Complete Beginner Guide - CppCalculator Project

> **For:** New team members with zero experience
> **Goal:** From nothing to running tests
> **Time:** ~45 minutes

Table of Contents

1. What You'll Build
2. Install Software
3. Verify Installation
4. Get the Project
5. Open in VS Code
6. Build the Project
7. Run the Program
8. Run the Tests
9. Understand the Code
10. Write Your Own Test
11. Common Problems
12. Quick Reference Card

1. What You'll Build

A **Calculator** program that can:

- Add, Subtract, Multiply, Divide
- Calculate Power and Square Root

You'll also learn to:

- Write automated tests

- Use mocking for testing

End result: A working program with 36 automated tests that verify everything works correctly.

2. Install Software

You need to install **4 things**. Follow each step exactly.

Step 2.1: Install VS Code (Code Editor)

What: A free code editor made by Microsoft.

1. Go to: https://code.visualstudio.com/
2. Click the big blue **"Download for Windows"** button
3. Run the downloaded file (`VSCodeUserSetup-x.x.x.exe`)
4. Click **Next** on all screens (accept defaults)
5. Click **Install**
6. Click **Finish**

Verify: You should see VS Code in your Start Menu.

Step 2.2: Install Git (Version Control)

What: A tool to download and manage code.

1. Go to: https://git-scm.com/download/win
2. Download will start automatically
3. Run the downloaded file
4. Click **Next** on all screens (accept all defaults)
5. **IMPORTANT:** When you see "Adjusting your PATH environment", select:
   - **"Git from the command line and also from 3rd-party software"** (default option)
6. Click **Next** until finish

Verify: Open Command Prompt (type `cmd` in Start Menu) and type:

```
git --version
```

You should see: `git version 2.x.x`

Step 2.3: Install MinGW (C++ Compiler)

What: The tool that converts your code into a program.

1. Go to: https://www.mingw-w64.org/
2. Click **"Downloads"**
3. Scroll down to **"MinGW-W64 Build Tools"**
4. Click the link for **"MinGW-W64 GCC-xx.x.x"** (latest version)
5. Run the downloaded file
6. In the installer:
   - **Version:** Choose latest
   - **Architecture:** Select `x86_64` (64-bit)
   - **Threads:** Select `posix`
   - **Exception:** Select `seh`
   - **Build revision:** Leave as default
7. Click **Next**
8. **IMPORTANT:** Note the installation folder (usually `C:\MinGW`)
9. Click **Install**
10. Click **Finish**

Add to PATH (CRITICAL STEP):

1. Press `Windows Key + R`
2. Type `sysdm.cpl` and press Enter
3. Click **"Advanced"** tab
4. Click **"Environment Variables"** button
5. Under **"User variables"**, select **"Path"** and click **"Edit"**
6. Click **"New"**
7. Type: `C:\MinGW\bin`
8. Click **OK** on all dialogs

Verify: Open NEW Command Prompt (close old one first) and type:

```
g++ --version
```

You should see: `g++.exe (GCC) x.x.x`

Step 2.4: Install CMake (Build Tool)

What: Helps build the testing framework.

1. Go to: https://cmake.org/download/
2. Download **"Windows x64 Installer"**
3. Run the downloaded file
4. Click **Next**, accept license
5. **IMPORTANT:** Select **"Add CMake to system PATH"** option
6. Click **Install**
7. Click **Finish**

Verify: Open NEW Command Prompt and type:

```
cmake --version
```

You should see: `cmake version 3.x.x`

3. Verify Installation

Open Command Prompt and run these commands one by one:

```
g++ --version
```

Expected: `g++.exe (GCC) 11.2.0` or similar

```
git --version
```

Expected: `git version 2.x.x`

```
cmake --version
```

Expected: `cmake version 3.x.x`

```
mingw32-make --version
```

Expected: `GNU Make 4.x`

If all commands show version numbers, you're ready! If any fails, go back to Step 2.

4. Get the Project

Step 4.1: Clone the Project

Open Command Prompt and run:

```
cd C:\
git clone https://github.com/your-repo/CppCalculator.git
cd CppCalculator
```

Note: If you already have the project folder, just navigate to it:

```
cd C:\CppCalculator
```

Step 4.2: Verify Project Structure

Run:

```
dir
```

Expected output:

```
 Directory of C:\CppCalculator

.vscode/
inc/
src/
obj/
bin/
lib/
tests/
third_party/
Makefile
SETUP_GUIDE.md
COMPLETE_BEGINNER_GUIDE.md
```

If you see these folders, the project is ready!

5. Open in VS Code

Step 5.1: Launch VS Code

```
code .
```

Note: The `.` means "current folder". This opens VS Code in the project.

Step 5.2: Install Required Extensions

1. Press `Ctrl+Shift+X` (opens Extensions panel)
2. Search for: `C/C++`
3. Install **"C/C++"** by Microsoft (the first one)
4. Wait for installation to complete

Step 5.3: Trust the Folder

If you see "Do you trust the authors of the files in this folder?":

- Click **"Yes, I trust the authors"**

Step 5.4: Verify IntelliSense

1. Open `inc/calculator.h` (double-click in left panel)
2. You should NOT see red squiggly lines under `#include` statements
3. If you see red lines, press `Ctrl+Shift+P` and type `C/C++: Reset IntelliSense Database`

6. Build the Project

Step 6.1: Build Using VS Code

1. Press `Ctrl+Shift+B`
2. You should see output in the terminal at bottom:

```
g++ -Wall -Wextra -g -I inc -std=c++17 -c src/calculator.cpp -o obj/calculator.o
g++ -Wall -Wextra -g -I inc -std=c++17 -c src/main.cpp -o obj/main.o
g++  obj/calculator.o obj/main.o -o bin/calculator.exe
```

Verify: Check that `bin/calculator.exe` now exists.

Step 6.2: Build Using Terminal (Alternative)

If `Ctrl+Shift+B` doesn't work, open terminal in VS Code (`Ctrl+`) and run:

```
mingw32-make all
```

Expected: Same output as above.

7. Run the Program

Step 7.1: Run Using VS Code

1. Press `F5`
2. A command window should open
3. You should see the calculator menu:

```
=== Calculator ===
1. Add
2. Subtract
3. Multiply
4. Divide
5. Power
6. Square Root
0. Exit
Choice:
```

4. Type `1` and press Enter
5. Type `5 3` (two numbers separated by space) and press Enter
6. You should see: `Result: 8`
7. Type `0` and press Enter to exit

Step 7.2: Run Using Terminal (Alternative)

```
bin\calculator.exe
```

8. Run the Tests

This is the main goal! Tests verify your code works correctly.

Step 8.1: Build and Run All Tests

Open terminal in VS Code (`Ctrl+`) and run:

```
mingw32-make test
```

Expected output:

```
g++ -Wall -Wextra -g -I inc -std=c++17 -I third_party/googletest/googletest/include -I third_par
g++  obj/CalculatorTest.o obj/calculator.o third_party/googletest/build/lib/libgtest.a third_par
./bin/calculator_test.exe
[==========] Running 36 tests from 3 test suites.
[----------] 28 tests from CalculatorTest
[ RUN      ] CalculatorTest.Add_PositiveNumbers
[       OK ] CalculatorTest.Add_PositiveNumbers (0 ms)
[ RUN      ] CalculatorTest.Add_NegativeNumbers
[       OK ] CalculatorTest.Add_NegativeNumbers (0 ms)
...
[  PASSED  ] 36 tests.
```

SUCCESS: If you see `[  PASSED  ] 36 tests.` , everything works!

Step 8.2: Run Tests in VS Code

1. Press `Ctrl+Shift+T`
2. Same output should appear

Step 8.3: Run Specific Tests

Run only Add tests:

```
mingw32-make test-add
```

Run only Mock tests:

```
mingw32-make test-mock
```

```
**Run with verbose output:**
```

mingw32-make test-verbose

```
---
```

```
## 9. Understand the Code
```

```
### Project Structure
```

CppCalculator/
├── inc/ # Header files (declarations)
│ ├── ICalculator.h # Interface (what Calculator can do)
│ ├── calculator.h # Calculator declaration
│ └── MockCalculator.h # Mock for testing
├── src/ # Source files (implementation)
│ ├── calculator.cpp # Calculator implementation
│ └── main.cpp # Program entry point
├── tests/ # Test files
│ └── CalculatorTest.cpp # All unit tests
├── third_party/ # External libraries
│ └── googletest/ # Testing framework
├── obj/ # Compiled files (don't touch)
├── bin/ # Executable files (generated)
├── lib/ # Libraries
└── Makefile # Build instructions

```
### Key Files Explained

**`inc/ICalculator.h`** - The Interface
```cpp
class ICalculator {
public:
    virtual ~ICalculator() = default;
    virtual double add(double a, double b) = 0;
    // ... other functions
};
```

This defines WHAT Calculator can do (the contract).

`inc/calculator.h` - The Declaration

```cpp
class Calculator : public ICalculator {
public:
    double add(double a, double b) override;
    // ... other functions
};
```

This declares the actual Calculator class.
```

`src/calculator.cpp` - The Implementation

```
double Calculator::add(double a, double b) {
    return a + b;
}
```

This is HOW each function works.

`tests/CalculatorTest.cpp` - The Tests

```
TEST_F(CalculatorTest, Add_PositiveNumbers) {
    EXPECT_DOUBLE_EQ(calc.add(2.0, 3.0), 5.0);
}
```

This VERIFIES the code works correctly.

10. Write Your Own Test

Let's add a simple test to learn the process.

Step 10.1: Open Test File

Open `tests/CalculatorTest.cpp`

Step 10.2: Add Your Test

Scroll to the end (before the `main()` function) and add:

```
TEST_F(CalculatorTest, MyFirstTest) {
    // Test that 10 + 5 = 15
    EXPECT_DOUBLE_EQ(calc.add(10.0, 5.0), 15.0);
}
```

Step 10.3: Run Tests Again

```
mingw32-make test
```

Expected: You should see your test in the output:

```
[ RUN      ] CalculatorTest.MyFirstTest
[       OK ] CalculatorTest.MyFirstTest (0 ms)
```

And total tests should be 37 (was 36).

Step 10.4: Try a Failing Test

Add another test:

```
TEST_F(CalculatorTest, MyFailingTest) {
    // This will FAIL because 10 + 5 != 20
    EXPECT_DOUBLE_EQ(calc.add(10.0, 5.0), 20.0);
}
```

Run tests again:

```
mingw32-make test
```

Expected: You'll see a failure:

```
[ RUN      ] CalculatorTest.MyFailingTest
path/to/CalculatorTest.cpp:XXX: Failure
Expected: calc.add(10.0, 5.0)
  Which is: 15
To be equal to: 20.0
[  FAILED  ] CalculatorTest.MyFailingTest (0 ms)
```

This is good! It shows the test framework catches bugs.

Step 10.5: Fix the Test

Change the expected value back to 15.0:

```
TEST_F(CalculatorTest, MyFailingTest) {
    EXPECT_DOUBLE_EQ(calc.add(10.0, 5.0), 15.0);
}
```

Run tests - all should pass now!

11. Common Problems

Problem 1: "g++ is not recognized"

Cause: MinGW not in PATH.

Fix:

1. Open Command Prompt
2. Run: `setx PATH "%PATH%;C:\MinGW\bin"`
3. Close and reopen Command Prompt
4. Try again

Problem 2: "make is not recognized"

Cause: MinGW/bin not in PATH.

Fix: Same as Problem 1.

Problem 3: Build fails with "No such file or directory"

Cause: Wrong directory.

Fix:

```
cd C:\CppCalculator
dir
```

Verify you see `Makefile` in the list. If not, `cd` to the correct folder.

Problem 4: Red squiggly lines in VS Code

Cause: IntelliSense not configured.

Fix:

1. Press `Ctrl+Shift+P`

2. Type: `C/C++: Reset IntelliSense Database`
3. Press Enter

Problem 5: "Permission denied" when running tests

Cause: Antivirus blocking.

Fix:

1. Open Windows Security
2. Go to "Virus & threat protection"
3. Add exclusions for `C:\CppCalculator`

Problem 6: Tests don't run

Cause: Test executable not built.

Fix:

```
mingw32-make clean
mingw32-make test
```

Problem 7: "TabError: inconsistent use of tabs and spaces"

Cause: Makefile has spaces instead of tabs.

Fix:

1. Open Makefile in VS Code
2. Enable: View → Render Whitespace
3. Ensure recipe lines start with TAB, not spaces
4. Or just delete and copy the Makefile from the project

Problem 8: Wrong number of tests

Cause: Compilation cache issue.

Fix:

```
mingw32-make clean
mingw32-make test
```

12. Quick Reference Card

Essential Commands

Action	Command	Shortcut
Build	`mingw32-make all`	`Ctrl+Shift+B`
Run	`bin\calculator.exe`	`F5`
Run tests	`mingw32-make test`	`Ctrl+Shift+T`
Clean	`mingw32-make clean`	-
Rebuild all	`mingw32-make clean && mingw32-make test`	-

Test Commands

Command	Description
`mingw32-make test`	Run all tests
`mingw32-make test-add`	Run only Add tests
`mingw32-make test-mock`	Run only Mock tests
`mingw32-make test-verbose`	Run with details

VS Code Shortcuts

Shortcut	Action
`Ctrl+Shift+B`	Build project
`F5`	Start debugging
`Ctrl+Shift+T`	Run tests
`` `Ctrl+``	Toggle terminal
`Ctrl+Shift+X`	Extensions

Shortcut	Action
`Ctrl+Shift+P`	Command palette

File Locations

File	Purpose
`inc/*.h`	Header files
`src/*.cpp`	Source files
`tests/*.cpp`	Test files
`bin/*.exe`	Executables
`obj/*.o`	Compiled objects
`Makefile`	Build instructions

Test Assertions

Assertion	Use For
`EXPECT_EQ(a, b)`	Equality
`EXPECT_DOUBLE_EQ(a, b)`	Floating point
`EXPECT_THROW(stmt, Exc)`	Exception
`EXPECT_TRUE(cond)`	Boolean
`ASSERT_*`	Abort on failure

Success Checklist

Before you start working, verify all these:

- [ ] `g++ --version` shows version
- [ ] `git --version` shows version
- [ ] `cmake --version` shows version

- [ ] `mingw32-make --version` shows version
- [ ] VS Code opens with `code .`
- [ ] `Ctrl+Shift+B` builds successfully
- [ ] `F5` runs the calculator
- [ ] `mingw32-make test` shows `[  PASSED  ] 36 tests.`

If all boxes are checked, you're ready to work!

Need Help?

1. Check Common Problems section
2. Ask your team lead
3. Check the original `SETUP_GUIDE.md` for advanced topics

Guide version: 1.0 | Last updated: July 2026